

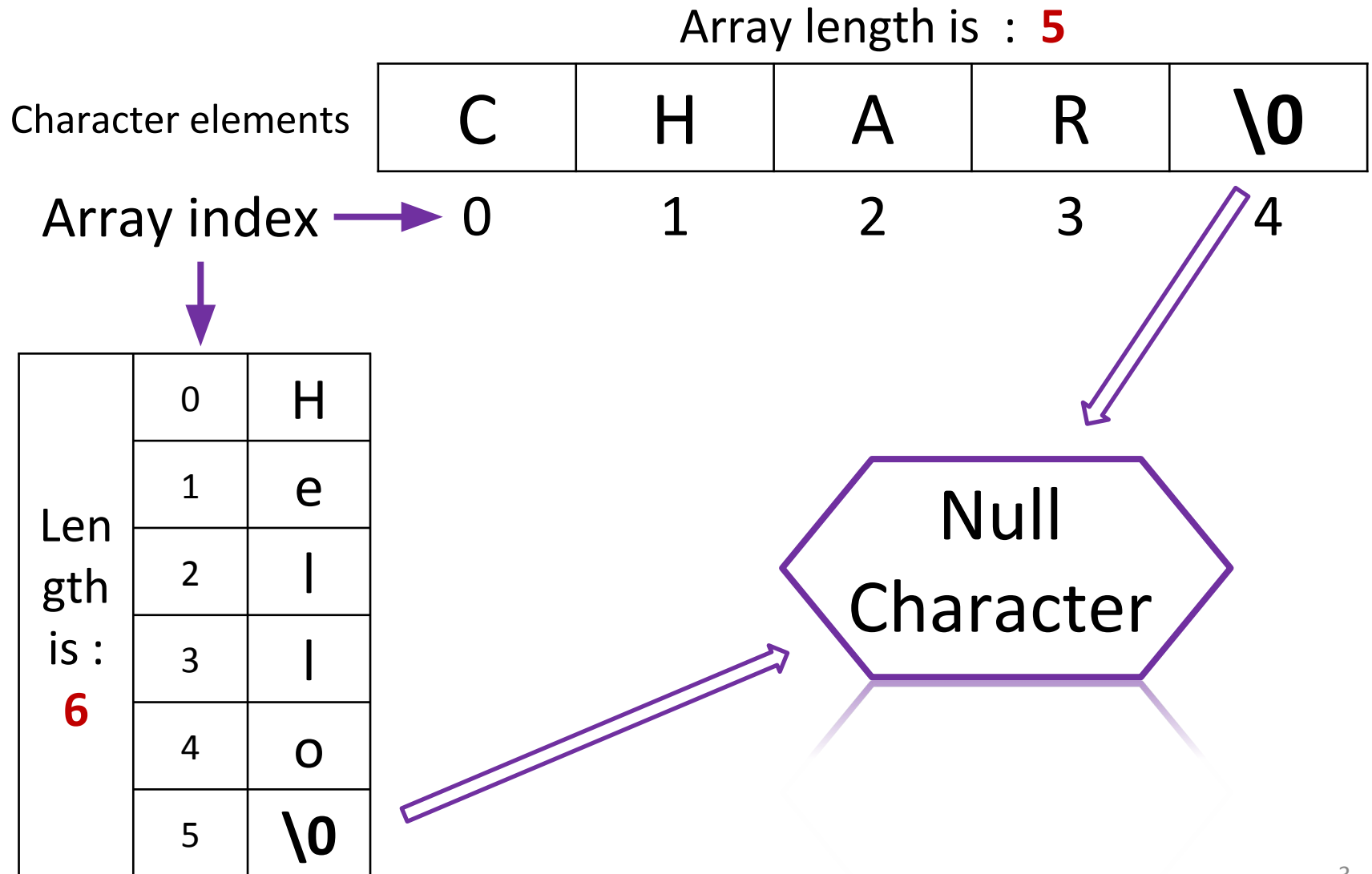
**M
OD
UL
E 2**

Strings

Character array

- A Character array (String) is a derived data type - to store a collection of characters.
- A char data type takes 1 byte of memory
- The null character `\0` is used to find the end of characters in the array - stored in the index after the last character

Representation of String



Why `\0` – null character for terminating string ?

- String is not a data type in C
- It is a variable-length structure stored in a fixed length array
- We need some way to determine the end of the string. So `\0` serves as “end-of-string” marker
- The character `'\0'` is guaranteed to be a byte with all bits - **zero**

Note: `string` is the **pointer** to the character array where the input string is stored

String - Declaration syntax

```
char string_name[size];
```

Example

```
char name[20];
```

```
char text[100];
```

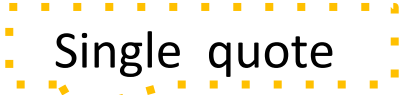
```
char str[15];
```

String- Initialization syntax

Character Array - `char string_name[size] = {list of string};`

Example

`char name[6] = { 'I', 'n', 'd', 'i', 'a', '\0' } ;`



Single quote

`char name[] = "Ram";`



Double quote

Note: Initialization of NULL character is not essential.
The C compiler inserts the NULL character automatically

WAP to print Welcome

```
#include <stdio.h>
int main()
{
char str1[8] = {"Welcome"};
char str2[8]={ 'W','e','l','c','o','m','e'};
char str3[7] = {"Welcome"};
printf("%s \n", str1);
printf("%s \n", str2);
printf("%s \n", str3);
return 0;
}
```

Error due to less size 

Output:

```
Welcome
Welcome
WelcomeWelcome
```

WAP to accept the string from user and display the string

```
#include<stdio.h>
int main()
{
char word[100];
printf("Enter the word : ");
scanf("%s", word);
printf("The given word is : %s", word);
return 0;
}
```

Output:
Enter the word : Model
The given word is : Model

WAP to print element of string using while loop

```
#include<stdio.h>
int main()
{
    int i;
    char word[13] = "Good Morning";
    while(i<13)
    {
        printf("%c", word[i]);
        i++;
    }
    return 0;
}
```

Or
while(word[i] != '\0')

Output:
Good Morning

Reading / writing a string

```
char address[10];
```

```
scanf("%s", address);
```

```
*****
```

```
char word_1[40], word_2[20];
```

```
printf("Enter two words :");
```

```
scanf("%s %s", word_1, word_2);
```

```
printf("%s", word_1);
```

Note: For character array(string), '**&**' is NOT required before scanf. str is already pointing to first element in char array

String handling functions

<string.h>

Function Name	Function Description
<code>strlen()</code>	Returns the length of the string.
<code>strcpy()</code>	Copy one string to another.
<code>strcat()</code>	Concatenates two strings.
<code>strcmp()</code>	Compares two strings.

Built-in string functions

Function	Description	Syntax
<code>strlen()</code>	Find the length of a string excluding '\0' NULL character.	strlen (str);
<code>strcpy()</code>	Copies a string from the source to the destination.	strcpy (dest, src);
<code>strncpy()</code>		strncpy (dest, src, n);
<code>strcat()</code>	Concatenate one string to the end of another.	strcat (dest, src);
<code>strncat()</code>		strncat (dest, src, n);

Built-in string functions

Function	Description	Syntax
<code>strcmp()</code>	Compares these two strings lexicographically.	<code>strcmp(s1, s2);</code>
<code>strncmp()</code>		<code>strncmp(s1, s2, n);</code>
<code>strchr()</code>	Find the first occurrence of a character in a string.	<code>strchr(s, c);</code>
<code>strrchr()</code>	Find the last occurrence of a character in a string.	<code>strchr(s, ch);</code>

strlen()

Syntax: strlen(string)

```
#include<stdio.h>
#include<string.h>
int main()
{
    int length_string;
    char word[100];
    printf("Enter the word : ");
    scanf("%s", word);
    length_string = strlen(word);
    printf("\n The given word is : %s", word);
    printf(" \n The length of the given string is :
%d",length_string);
    return 0;
}
```

Output - strlen()

Output 1:

Enter the word : **Welcome**

The given word is : **Welcome**

The length of the given string is : **7**

Note: Null character
is not counted !

Output 2:

Enter the word : **Hello World**

The given word is : **Hello**

The length of the given string is : **5**

The problem with scanf:
It **terminates** its input on the first WHITE
SPACE.

strcat()

- It will append a copy of the source string to the end of the destination string.

strncat()

- This function appends not more than *n* characters from the string pointed to by source to the end of the string pointed to by destination plus a terminating NULL character.

strcat()

```
#include<stdio.h>
#include<string.h>
int main()
{
    char word1[50]= "Hello" ;
    char word2[50]= "World";
    printf("First Word is : %s\n", word1);
    printf("Second Word is : %s\n", word2);
    strcat(word1,word2) ;
    printf("\n The concatenated word is : %s", word1);
    return 0;
}
```

Output:

First Word is : Hello

Second Word is : World

The concatenated word is :
HelloWorld

String concat with space

```
#include<stdio.h>
#include<string.h>
int main()
{
    int i, len1, len2;
    char word1[30], word2[10];
    puts("Enter first word: ");
    scanf("%s", word1);

    puts("Enter second word: ");
    scanf("%s", word2);
    strcat(word1, " ");
    strcat(word1, word2);
    printf("%s\n", word1);
    return 0;
}
```

Ouput:

```
Enter the first word : Hello
Enter the second word : World
Hello World
```

strcpy()

It will assign the contents of string2 to string1.

Syntax : strcpy(string1, string2);

```
char str1[20];  
char str2[20] = "C programming";  
strcpy(str1, str2);  
puts("The copied string1 is : ");  
puts(str1);  
puts("The retained string2 is : ");  
puts(str2);
```

Output - strcpy()

The copied string1 is : **C programming**

The retained string2 is : C programming

```
char str1[20] = "Welcome";  
char str2[20] = "C programming";  
strcpy(str1, str2);
```

The copied string1 is : **C programming**

The retained string2 is : C programming

Note : The string 1 is overwritten by string2

strcmp()

It will compare 2 strings; if equal , gives value= 0

Syntax : strcmp(string1, string2);

```
int result1, result2, result3;
char word1[] = "India";
char word2[] = "INDIA";
char word3[] = "INDIAN";
char word4[] = "INDIAN";
result1 = strcmp(word1, word2);
result2 = strcmp(word2, word3);
result3 = strcmp(word3, word4);
printf("The result 1 is : %d\n", result1);
printf("The result 2 is : %d\n", result2);
printf("The result 3 is : %d\n", result3);
```

Output - strcmp()

The result 1 is : 32 (Diff betwn 'n' and 'N' = 110-78 = 32)

The result 2 is : -78 (Diff betwn '\0' and 'N' = 0-78 = -78)

The result 3 is : 0

I	n	d	i	a	\0	
I	N	D	I	A	\0	
I	N	D	I	A	N	\0
I	N	D	I	A	N	\0

Note: Refer to ASCII code

strcspn ()

Full form : **String Complement Span**

- The strcspn() function searches for the first occurrence in a string of any of the specified characters and returns the length of the string up to that point.
- If **none** of the characters are found then the **length of the string is returned**.
- Computes the **length of the initial segment** of a string that **does not contain** any of the characters from another given string

strcspn ()

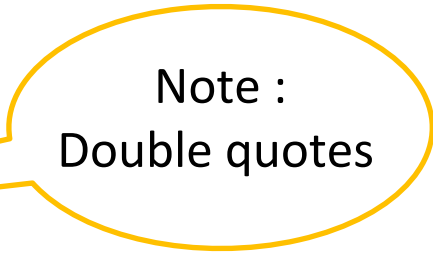
Syntax

```
strcspn(char str1, char str2)
```

WAP to check first Occurrence of a char

```
#include <stdio.h>
#include <string.h>
int main()
{
    int pos;
    char str[] = "Hello World!";
    char charToFind[] = "o";
    pos = strchr(str, charToFind[0]);
    printf("First occurrence of 'o' is at
           position : %d\n", pos);

    return 0;
}
```

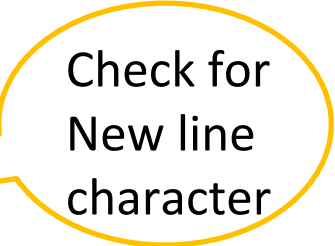


Note :
Double quotes

Output:
The first occurrence of 'o' is at position : 4

WAP to find the length of a sentence using **strcspn**

```
#include <stdio.h>
#include <string.h>
int main()
{
    int len;
    char str[] = "My name is Billa";
    char charToFind[] = "\n";
    len = strcspn(str, charToFind);
    printf("The length of the line is:%d", len);
    return 0;
}
```



Output:
The length of the line is: 16

<ctype.h>

Function Name	Function Description
<code>islower(c)</code>	Determines if the argument is lowercase
<code>isupper(c)</code>	Determines if the argument is uppercase
<code>tolower(c)</code>	Converts letter to lower case
<code>toupper(c)</code>	Converts letter to upper case

WAP to reverse a string (word) without using string handling function strrev()

```
char string[20], temp;  
int i, length, half_length;  
printf("Enter String : ");  
scanf("%s", string);  
length = strlen(string);  
half_length = length/2;  
for(i=0; i<half_length; i++)  
{  
    temp = string[i];  
    string[i] = string[length-1-i];  
    string[length-1-i] = temp;  
}  
printf("The Reverse string :%s", string);
```

Output:
Enter String : Welcome
The Reverse string :
emocleW

gets(str) is vulnerable ?

- The **gets()** function is unsafe to use due to its vulnerability to *buffer overflow* attacks.
- Buffer overflow happens when more data is written to a buffer (temporary storage area in memory) than it can hold.
- If the user inputs a string that is larger than the allocated memory size of the input buffer, the excess data will **overwrite the adjacent memory** locations, potentially overwriting important data

fgets()

- Syntax:

```
fgets(char *str, int n, FILE *stream);
```

Example

```
char buffer[50];
```

```
    fgets(buffer, 50, stdin)
```



Standard input

- We can both read text lines from a file or the standard input stream by using the `fgets` function

Difference between `gets` and `fgets`

Aspect	<code>gets()</code>	<code>fgets()</code>
Buffer Size Control	No size control so may lead to buffer overflow.	Allows size control preventing buffer overflow.
Newline Handling	Discards newline character.	Retains newline character.
Input Source	Can read from stdin only.	Can read from any input stream including stdin.
Error Handling	Cannot detect errors or EOF so no way to handle read failures.	Returns NULL on error or EOF so can handle read failure efficiently.
Status	Deprecated in C11 and later.	Recommended and widely used.

puts ()

Syntax : `puts (char* str) ;`

- It prints strings character by character until the NULL character is encountered.
- The `puts()` function prints the **newline** character at the end of the output string.

Note: `'\n'` in `printf` is not needed in `puts`

fgets , printf, puts

```
#include <stdio.h>
int main() {
    char str[50];
    printf("Enter a string: ");
    fgets(str, 50, stdin);
    printf("You entered: %s\n", str);
    return 0;
}
```

Enter a string: Hello World
You entered: Hello World

```
#include <stdio.h>
int main() {
    char str[50];
    puts("Enter a string: ");
    fgets(str, 50, stdin);
    puts("You entered: ");
    puts(str);
    return 0;
}
```

Enter a string:
Hello World
You entered:
Hello World

fgets , puts

```
#include <stdio.h>
```

```
int main() {
```

```
    char str[100];
```

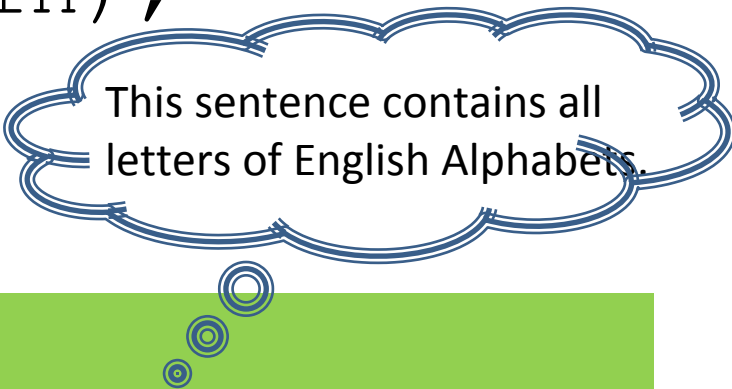
```
    puts("Enter a string: ");
```

```
    fgets(str, 100, stdin);
```

```
    puts(str);
```

```
    return 0;
```

```
}
```



This sentence contains all
letters of English Alphabets.

Enter a string:

The quick brown fox jumps over the lazy dog.

The quick brown fox jumps over the lazy dog.

WAP to find the number of digits, character and space in string

```
#include <stdio.h>
int main()
{
char addr[100];
int i, digits=0, alph =0, spaces = 0;
puts("Enter address :");
fgets(addr,100,stdin);
for(i=0;addr[i]!='\0'; i++)
{
    if ((addr[i]>='a' && addr[i]<='z') ||
        (addr[i] >='A') && (addr[i]<='Z'))
        alph = alph + 1;
}
```

WAP to find the number of digits, character and space in string

```
else if ((addr[i] >= '0') && (addr[i] <= '9'))
    digits = digits + 1;
else
    spaces = spaces + 1;
}
printf("No. of alph = %d \n", alph);
printf("No. of digits = %d \n", digits);
printf("No. of spaces = %d \n", spaces);
return 0;
}
```

Output

Enter address :

House No. 10; GandhiNagar 2nd street

No. of alph = 26

No. of digits = 3

No. of spaces = 8

WAP to count number of vowel, space, consonant

```
#include <stdio.h>
int main()
{
char line[100];
int i, vowel=0, spaces =0, consonant = 0;
puts("Enter a sentence :");
fgets(line,100,stdin);
for(i=0;line[i]!=0; i++)
{
if ((line[i]== 'a') || (line[i]== 'e') || (line[i]==
        'i') || (line[i]== 'o') || (line[i]== 'u'))
    vowel = vowel + 1;
```

Also consider capital
vowels

WAP to count number of vowel, space, consonant

```
else if (line[i] == ' ')
    spaces = spaces + 1;
else
    consonant = consonant + 1;
}
printf("No. of vowel = %d \n", vowel);
printf("No. of spaces = %d \n", spaces);
printf("No. of consonant = %d \n",
        consonant);
return 0;
}
```


Output

Enter a sentence :

The quick brown fox jumps over the lazy dog

No. of vowel = 11

No. of spaces = 8

No. of consonant = 25

KTU- Model Qn No. 12 b

Write a C program that allows the user to enter a line of text, store it in an array, and then **display it backward**. The line length should be unspecified (terminated by pressing the Enter key), but assume it will not exceed 80 characters.

Display sentence backwards

```
#include <stdio.h>
#include <string.h>
int main()
{
    char text[1000];
    int i,temp,len;
    printf("Enter a line in text: \n");
    fgets(text, sizeof(text), stdin);
    len = strlen(text);
    puts("The reverse of the text is : ");
    for (i = 0; i < len; i++)
    {
        printf("%c", text[len-i-1]);
    }
    return 0;
}
```

Output

Enter a line in text:

Your success and happiness lie in you.

The reverse of the text is :

.uoy ni eil ssenippah dna sseccus ruoY

Reverse a sentence

```
#include <stdio.h>
#include <string.h>
int main()
{
char text[1000];
int i, temp, len;
printf("Enter a line in text: ");
fgets(text, sizeof(text), stdin);
len = strlen(text);
printf("The length of the text is: %d\n", len);
```

Reverse a sentence - contd

```
for (i=0; i < len/2; i++)
{
    temp = text[i];
    text[i] = text[len-i-1];
    text[len-i-1] = temp;
}
puts("The reverse of the text is : ");
puts(text);
return 0;
}
```

Output

Enter a line in text:

Mistakes are proof that you are trying.

The length of the text is: **40**

The reverse of the text is :

.gniyrT era uoy taht foorp era sekatsiM

Programs

- WAP to check if a character is present in a string
- WAP to concatenate two strings without string handling function
- WAP to find the length of the string without string handling function
- WAP accept a string from the user. Display the count of upper case and lowercase characters in that string.

Array of strings

2-D strings

Array of Strings (2-D)

- An array of strings is a 2D array where each row contains a sequence of characters terminated by a '\0' NULL character (strings).
- It is used to store multiple strings in a single array.
- While a string is a 1-D array of characters terminated with the null character, the array of strings will evolve into a 2-D array of characters.

Declaration syntax of string array

```
char    arr_str_name[No_of_strings]
                               [MaxCharof_eachString];
```

Example

```
char Student[30][18]
```

This array is used to store a list of **30 names** of students ,
each of length not more than **18 characters**

Initialization of string array

Example

```
char car[4][20] = {"Mahindra",  
"Audi", "Lamborghini", "Hyundai"};
```

```
char sports[5][10] = { "golf",  
"hockey", "football", "cricket",  
"shooting" };
```

Memory representation of array of strings

```
sports[5][10]
```

1000	g	o	l	f	\0	\0	\0	\0	\0	\0
1010	h	o	c	k	e	y	\0	\0	\0	\0
1020	f	o	o	t	b	a	l	l	\0	\0
1030	c	r	i	c	k	e	t	\0	\0	\0
1040	s	h	o	o	t	i	n	g	\0	\0

WAP to print an array of given string

```
#include<stdio.h>
int main()
{
    int i;

    char array_str[4][15] = {"Ram", "Sam", "Ann", "Liz"};
    printf("The strings are:\n");
    for(i=0;i<4;i++)
    {
        printf("%s\n", array_str[i]);
    }
    return 0;
}
```

Note: Accessing an item in 2-D array of string is similar to 1-D

Output:
The strings are:
Ram
Sam
Ann
Liz

WAP to print the array of strings from user

```
#include<stdio.h>
int main()
{
    int i, N;
    char array_str[5][15];
    printf("Enter the number of strings : ");
    scanf("%d",&N);
    printf("Enter the %d strings : \n", N);
    for (i=0;i<N;i++)
    {
        scanf("%s", array_str[i]);
    }
}
```

WAP to print the array of strings from user - contd

```
printf("The strings are:\n");  
for(i=0;i<N;i++)  
{  
    printf("%s\n",array_str[i]);  
}  
return 0;  
}
```


Output

Enter the number of strings :

5

Enter the 5 strings :

January

February

March

April

May

The strings are:

January

February

March

April

May

Sort array of strings in lexicographical order

```
#include<stdio.h>
#include<string.h>
int main()
{
    int i, j, N;
    char temp[12], arr_str[12][20];
    printf("Enter the number of strings : ");
    scanf("%d", &N);
    printf("Enter the %d strings : \n", N);
    for (i=0; i<N; i++)
    {
        scanf("%s", arr_str[i]);
    }
}
```

Sort array of strings in lexicographical order - contd

```
for (i=0; i<N-1; i++)  
{  
    for (j=0; j<N-1-i; j++)  
    {  
        if(strcmp(arr_str[j],arr_str[j+1]) > 0)  
        {  
            strcpy(temp, arr_str[j]);  
            strcpy(arr_str[j], arr_str[j+1]);  
            strcpy(arr_str[j+1], temp);  
        }  
    }  
}
```

Sort array of strings in lexicographical order - contd

```
printf("The sorted strings are : \n");  
    for(i=0;i<N;i++)  
    {  
        printf("%20s", arr_str[i]);  
    }  
return 0;  
}
```

Output

Enter the number of strings :

10

Enter the 10 strings :

Chennai

Hyderabad

Bangalore

Bhopal

Patna

Mumbai

Itanagar

Lucknow

Shimla

Jaipur

The sorted strings are :

Bangalore

Bhopal

Chennai

Hyderabad

Itanagar

Jaipur

Lucknow

Mumbai

Patna

Shimla